

FEM data ≠ ND-arrays

- N-dimensional (ND) arrays are the backbone of many areas of computational science. In particular they underpin the entire field of machine learning.
- Why are they good? Nice abstraction, good to program with..., can optimise
- Data structures for the finite element method (FEM) do not conveniently fit into this paradigm. unstructured mesh... different topological entities... parallel..
- This prohibits the sort of development that we see in ML...
- **pyop3 is this missing abstraction**

Core concepts in pyop3

The pyop3 language

1. Motivation

- The application of compact computational kernels across a mesh (fig. 1) crops up all the time in continuum mechanics simulations.
- Writing this traversal code by hand quickly becomes difficult due to the need to support different PDEs, discretisations, hardware, etc.
- **A software abstraction that can automate the construction of the traversal code is needed.**

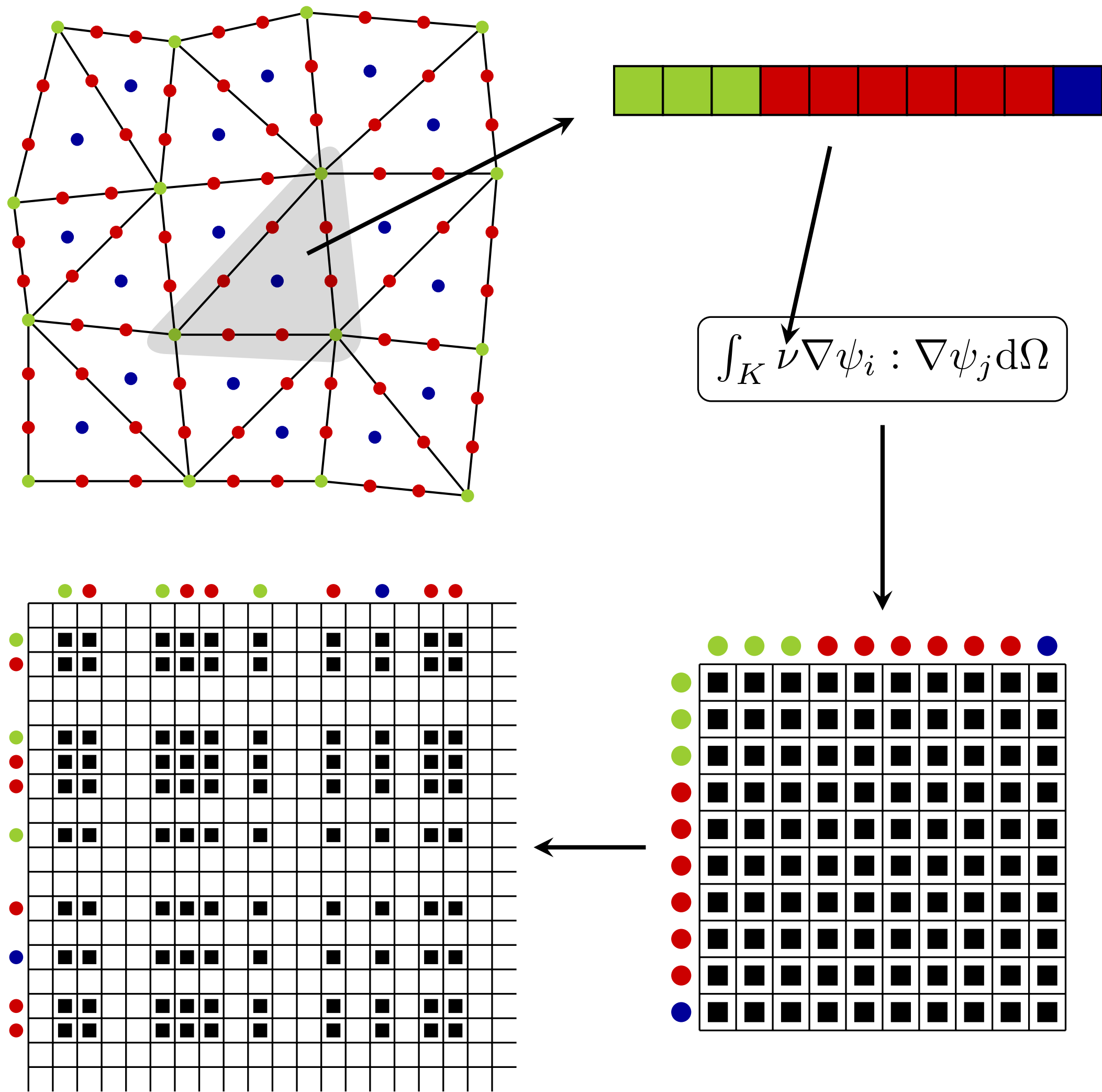


Figure 1: An example of applying a compact kernel to a mesh: finite element assembly of a matrix.

2. Introducing pyop3

- pyop3 is a new **domain-specific language** for automating calculations involving mesh traversal.

- It uses **code generation** (fig. 2) to outperform equivalent library interfaces.
- It is based on PyOP2 [**rathgeberPyOP2HighLevelFramework2012**], and work is in progress to replace PyOP2 with pyop3 in the Firedrake finite element framework [**FiredrakeUserManual**].

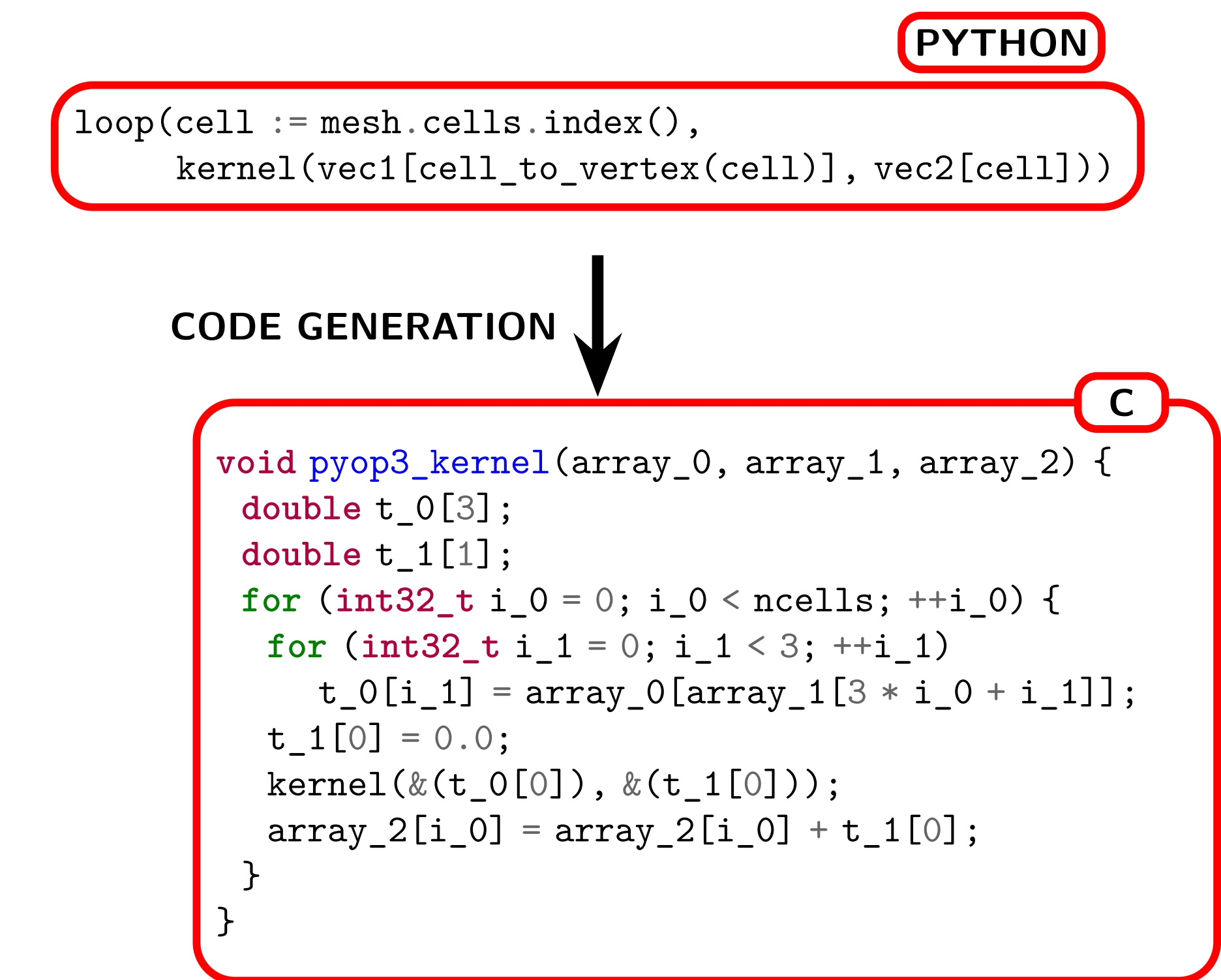


Figure 2: Simplified example of the code generated from a pyop3 loop expression.

References

3. A new abstraction for mesh data

- Mesh data has a complicated structure (fig. 3, fig. 4) that maps poorly to existing ways of describing data layouts, like N-dimensional arrays.
- Unlike PyOP2, pyop3 is capable of fully describing complex mesh data layouts using **axis trees** (fig. 5).
- Having a richer abstraction for data layouts makes composition easier, and thus pyop3 is able to describe and generate code for a wider range of problems than PyOP2.

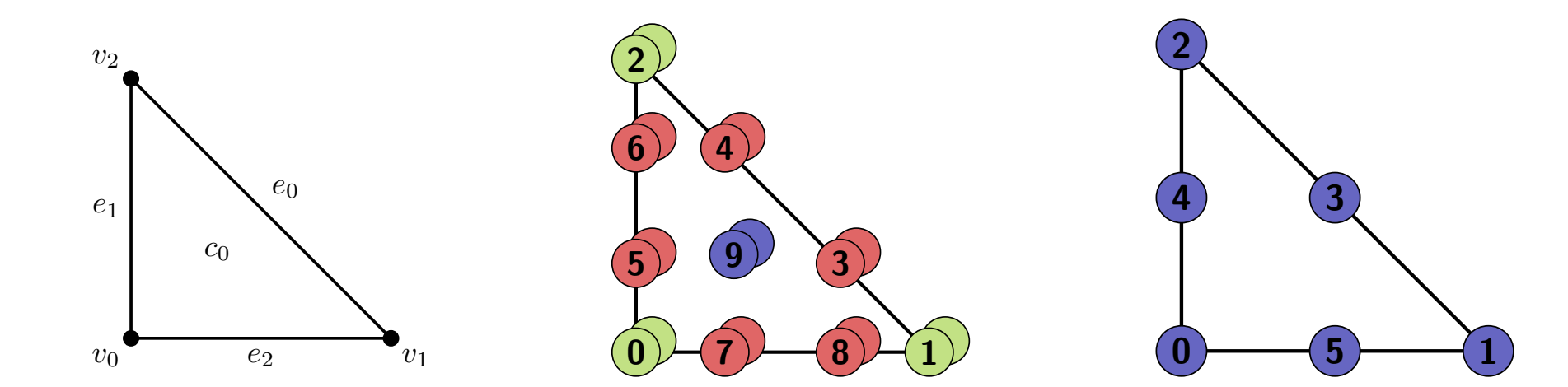


Figure 3: The Scott-Vogelius finite element pair $[P_3]^2 \oplus P_2^{\text{disc}}$ (middle and right) on a reference triangle (left). DoFs associated with vertices, edges and cells are shown in green, red and blue respectively. P_2^{disc} (right) is discontinuous, so all the DoFs are considered to belong to the cell.

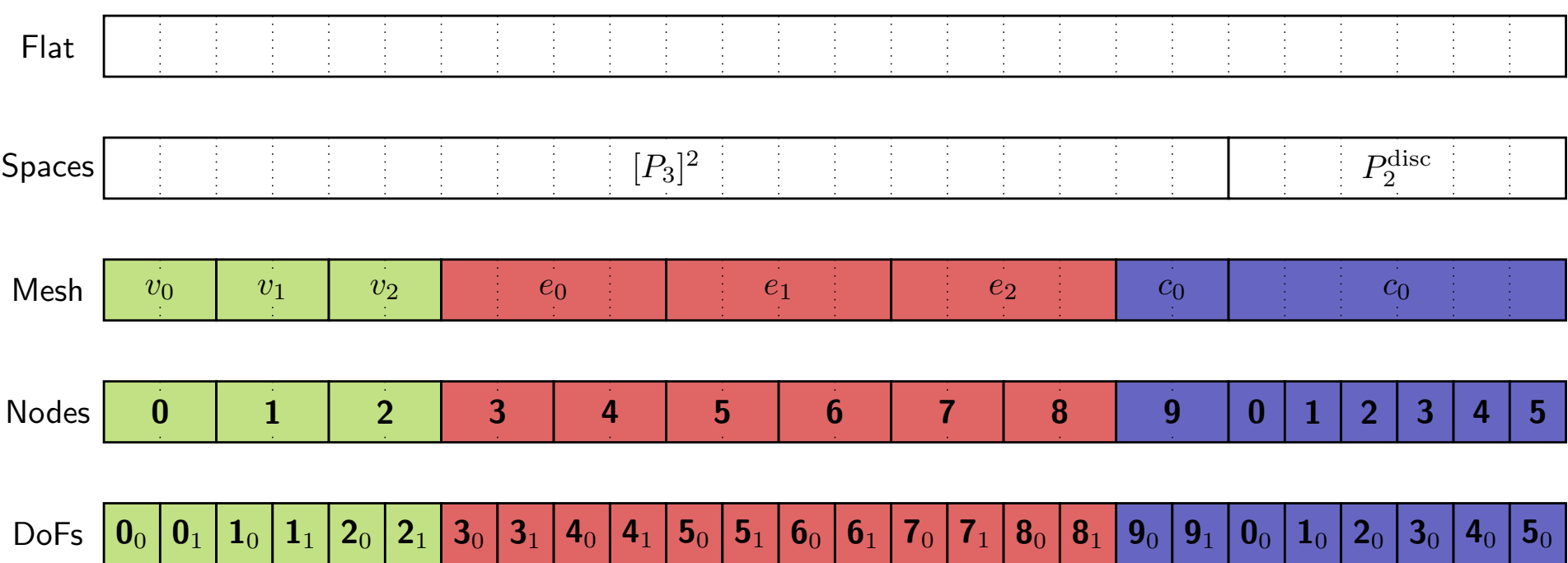


Figure 4: Different views of a possible data layout for a one-cell mesh of the finite element pair in fig. 3.

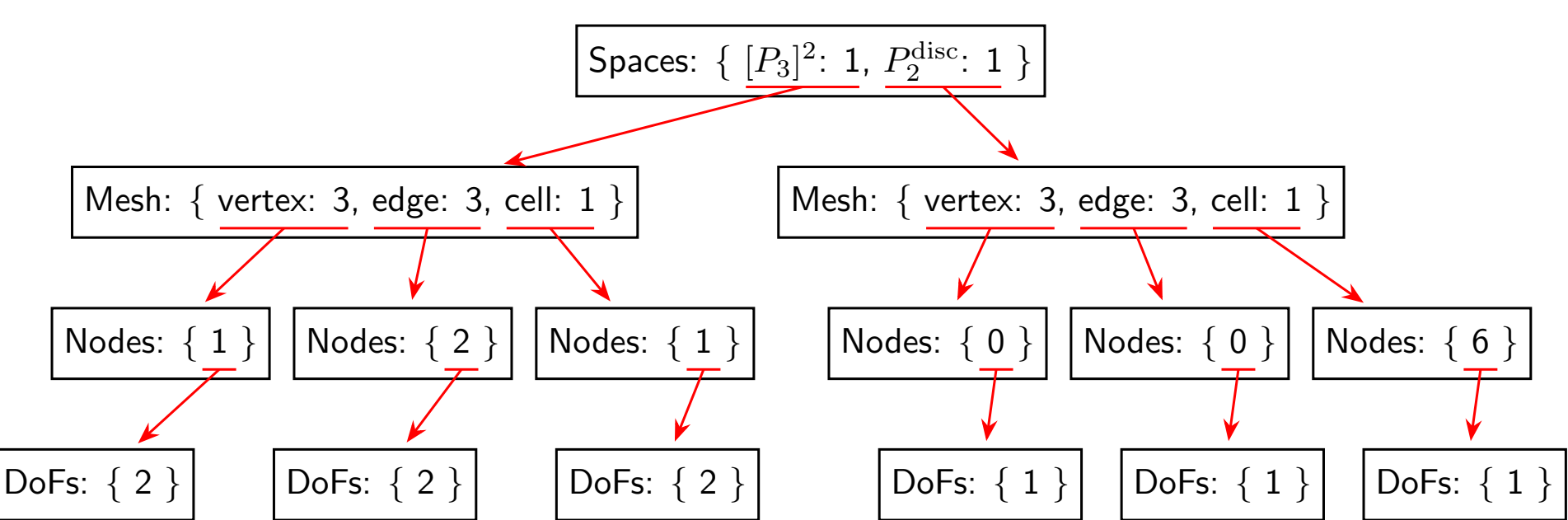


Figure 5: An axis tree that describes the data layout shown in fig. 4.

4. GPU support

4. Future work

Once pyop3 has been fully integrated into Firedrake there is scope for a variety of performance improvements and avenues of

research to pursue. Possible low-level changes include:

- Data layout transformations
- Loop tiling

Whilst at a high level the following should also become expressible:

- Physics-based preconditioners
- Slope limiters
- Adaptivity
- Particle methods
- [And more!](#)

